TD VHDL – Électronique Numérique

Initiation à VHDL

Notes et Conseils

Remarque : Ce TD constitue une introduction à la description matérielle en VHDL. Il est conseillé de travailler les exercices en amont de la séance et de tester les codes sur un simulateur (GHDL) pour valider vos acquis.

1 Premiers pas en VHDL

1. Dessiner l'enveloppe externe du composant décrit par cette entité.

Solution

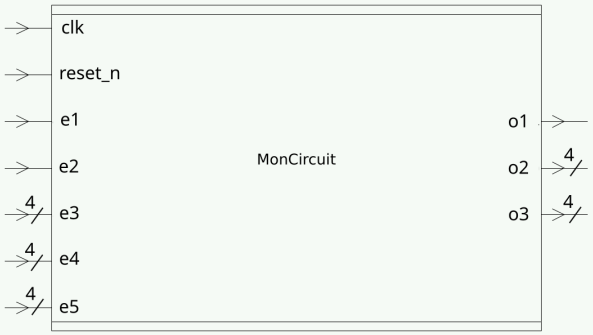


FIGURE 1 – Entity du composant MonCircuit

2. Dessiner l'entité d'un demi-additionneur, puis le code VHDL de cette entité.

Solution

Voir la figure 2.

3. Rappeler la constitution interne du demi-additionneur.

Solution

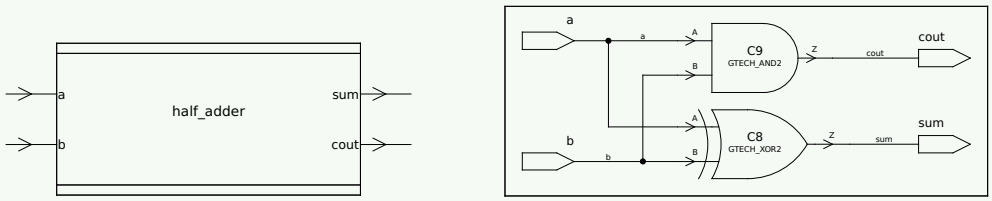


FIGURE 2 – Entity et architecture du demi-additionneur 1 bit

4. Coder l'architecture de ce demi-additionneur.

Solution

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity half_adder is
  port(
    a,b : in std_logic;
    sum,cout : out std_logic
  );
end half_adder;

architecture logic of half_adder is
begin
  sum <= a xor b;
  cout <= a and b;
end logic;
```

5. Dessinez la constitution interne d'un additionneur 1 bit complet, à partir du demi-additionneur.

Solution

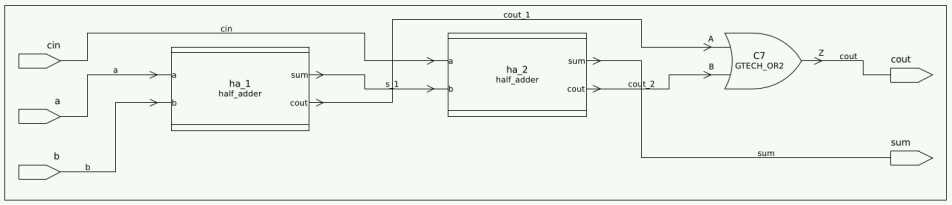


FIGURE 3 – Architecture de l'additionneur 1 bit complet

6. Coder en VHDL l'architecture de cet additionneur 1 bit.

Solution

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity full_adder is
  port(
    a,b : in std_logic;
    cin : in std_logic;
    sum,cout : out std_logic
  );
end full_adder;

architecture logic of full_adder is
  signal s_1, cout_1,cout_2 : std_logic;
begin

  ha_1 : entity work.half_adder(logic)
  port map(
    a => a,
    b => b,
    sum => s_1,
    cout => cout_1
  );

  ha_2 : entity work.half_adder(logic)
  port map(
    a => cin,
    b => s_1,
    sum => sum,
    cout => cout_2
  );

  cout <= cout_1 or cout_2;

end logic;
```

7. Coder en VHDL l'entité et l'architecture d'un additionneur 8 bits.

Solution

```
library ieee;
use ieee.std_logic_1164.all;

entity adder8b is
  port(
    a,b : in std_logic_vector(7 downto 0);
    sum : out std_logic_vector(7 downto 0);
    cout : out std_logic
  );
end adder8b;

architecture structural of adder8b is
  signal dummy_carry : std_logic;
  signal carry : std_logic_vector(7 downto 0);
begin

  dummy_carry <= '0';

  fa_0 : entity work.full_adder(logic)
  port map(
    a => a(0),
    b => b(0),
    cin => dummy_carry,
    sum => sum(0),
    cout => carry(0)
  );

  fa_1 : entity work.full_adder(logic)
  port map(
    a => a(1),
    b => b(1),
    cin => carry(0),
    sum => sum(1),
    cout => carry(1)
  );

  fa_2 : entity work.full_adder(logic)
  port map(
    a => a(2),
    b => b(2),
    cin => carry(1),
    sum => sum(2),
    cout => carry(2)
  );

  fa_3 : entity work.full_adder(logic)
  port map(
    a => a(3),
    b => b(3),
    cin => carry(2),
    sum => sum(3),
    cout => carry(3)
  );

  fa_4 : entity work.full_adder(logic)
  port map(
    a => a(4),
    b => b(4),
    cin => carry(3),
    sum => sum(4),
    cout => carry(4)
  );
```

2 Le banc de test (testbench)

1. Le testbench présente également une entité. Localisez cette entité. En quoi est-elle particulière ? Comment peut-on l'expliquer ?

Solution

Un tel banc de test possède une entité *vide* : elle ne possède aucune entrée, ni sortie. C'est le laboratoire, portes closes.

2. Un générateur d'horloge est contenu dans le banc de test. Combien de lignes sont nécessaires ? Dessinez le chronogramme de cette horloge.

Solution

Une seule ligne suffit. On inverse le signal tous les demi-périodes d'horloge, à l'aide d'une assignation conditionnée par un signal `running`, qui nous permettra de contrôler l'arrêt de la simulation.

3. Comment notre banc de test provoque-t-il concrètement cet arrêt ?

Solution

Le signal `running` est affecté à `false` à la fin du processus de stimulation. Plus aucun événement n'est créé par le simulateur.

4. Dessiner le chronogramme des stimuli.

Solution

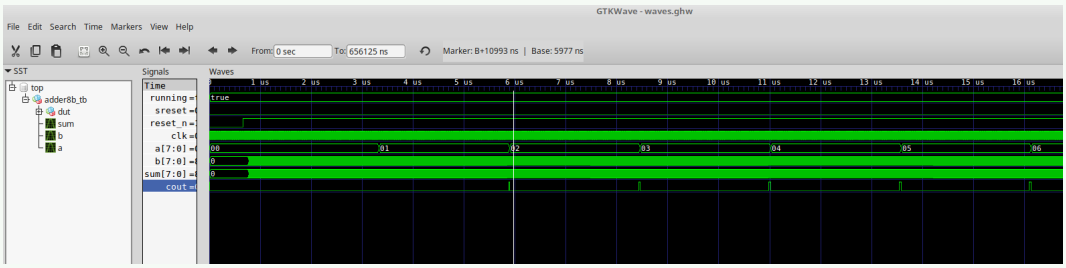


FIGURE 5 – Chronogramme (overview)

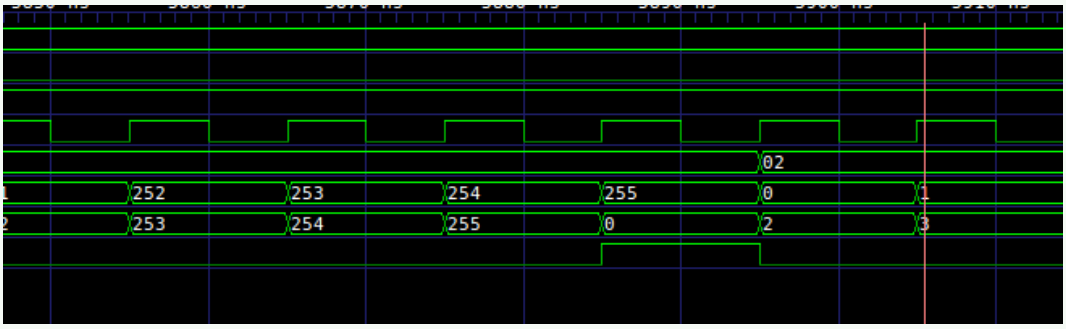


FIGURE 6 – Chronogramme : cas de $1 + 255$

5. L'additionneur fonctionne-t-il avec des nombres signés ?

Solution

Oui ! En tous les cas, un test semble aller en ce sens ! Voir la figure 7.

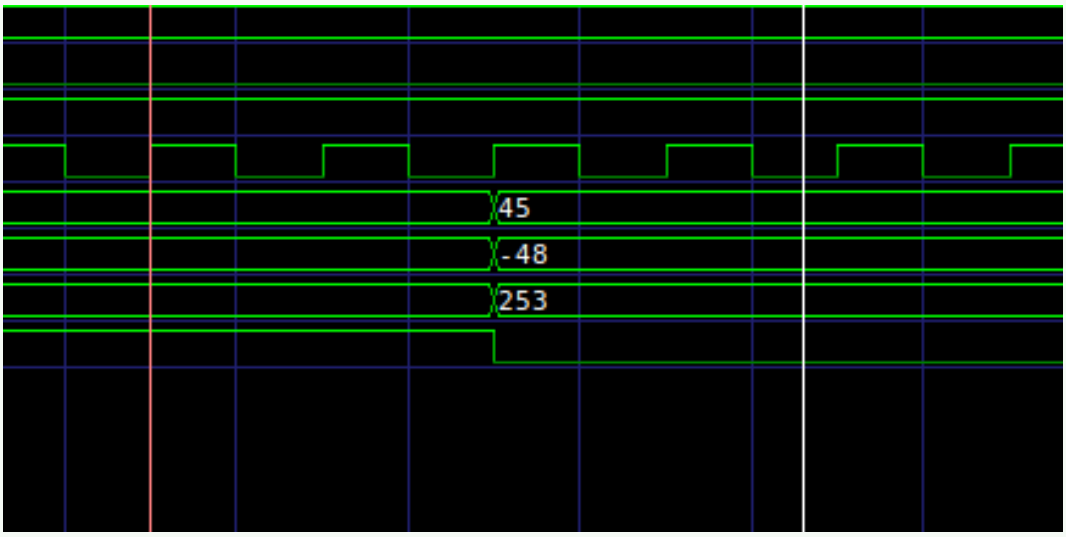


FIGURE 7 – L'additionneur fonctionne pour les nombres signés

3 Script de compilation

Solution

Script de compilation GHDL (donné sous Moodle) :

```
echo "=> cleaning..."
rm -rf waves.ghw *.o
echo "=> analyzing VHDL files..."
ghdl -a half_adder.vhd
ghdl -a full_adder.vhd
ghdl -a adder8b.vhd
ghdl -a adder8b_tb.vhd
echo "=> elaboration..."
ghdl -e adder8b_tb
echo "=> running simulation"
ghdl -r adder8b_tb --wave=waves.ghw
echo "=> starting waveform viewer"
gtkwave waves.ghw chrono.sav
```

Testbench (donné sous Moodle) :

```
-------------------------------------------------------------------
-- This file was generated automatically by vhdl_tb Ruby utility
-- date : (d/m/y) 05/11/2018 15:40
-- Author : Jean-Christophe Le Lann - 2014
-------------------------------------------------------------------
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity adder8b_tb is
end entity;

architecture bhv of adder8b_tb is

  constant HALF_PERIOD : time := 5 ns;

  signal clk : std_logic := '0';
  signal reset_n : std_logic := '0';
  signal sreset : std_logic := '0';
  signal running : boolean := true;

  procedure wait_cycles(n : natural) is
   begin
     for i in 1 to n loop
       wait until rising_edge(clk);
     end loop;
   end procedure;

  signal a : std_logic_vector(7 downto 0);
  signal b : std_logic_vector(7 downto 0);
  signal sum : std_logic_vector(7 downto 0);
  signal cout : std_logic;

begin
  -------------------------------------------------------------------
  -- clock and reset
  -------------------------------------------------------------------
  reset_n <= '0','1' after 666 ns;
  clk <= not(clk) after HALF_PERIOD when running else clk;

  -------------------------------------------------------------------
```

4 Utilisation de generate

Solution

L'instruction `generate` permet de créer des instances de composants de manière itérative, ce qui est très utile pour structurer des designs réguliers comme un additionneur N bits.

```
library ieee;
use ieee.std_logic_1164.all;

architecture using_generate of adder8b is
  signal dummy_carry : std_logic;
  signal carry : std_logic_vector(7 downto 0);
begin

  dummy_carry <= '0';

  Loop_index: for i in 0 to 7 generate

    first_stage: if i=0 generate
      stage_0: entity work.full_adder(logic)
      port map(
        a => a(i),
        b => b(i),
        cin => dummy_carry,
        sum => sum(i),
        cout => carry(i)
      );
    end generate;

    other_stages: if i>0 generate
      state_i: entity work.full_adder(logic)
      port map(
        a => a(i),
        b => b(i),
        cin => carry(i-1),
        sum => sum(i),
        cout => carry(i)
      );
    end generate;

  end generate;

  cout <= carry(7);

end using_generate;
```